The Ever-Agreeing Genie

Kent Beck calls his AI pair a genie: it grants wishes without judgment, exactly what you asked for, whether or not it is what you needed.[1] *The danger was never the genie. It was the wish.*

If a team builds understanding together, what keeps it honest?

I'm building a demo for my next talk on JoustMania again: OpenTelemetry and OpenFeature wired together into what I'm calling an agentic nervous system, a game that runs its own experiments over time and tracks what happened.

I got it working and watched it run.

And right behind the working demo came the doubt. Is this worth building, or does a framework already exist that does this better? Observability data is what the system already tells you about itself while it runs, so is it actually a sound foundation for the agent's experiments, or a convenient one I've talked myself into? I don't know which. The talk isn't even accepted yet. Nobody has seen it.

I research the answers with AI, and I notice what that research doesn't give me. It's fluent inside the architecture I hand it: ask whether the design is sound and it reasons about the design I described. Ask whether something already solves this and it searches for the thing I named, not the thing I didn't know to look for.

So I have a system that works, and a talk that might never get a stage. Either way, I still don't know if the shape of it is right.

There are four things you don't get from an AI, four things only another person can provide.

The first is *weighted validation.* When someone with their own hard-won experience, their own perspective, their own reasons to disagree, looks at what you're building and says "yes, that's

[1] Kent Beck named the metaphor first. See "Genie Wants to Leap," *Tidy First?* (tidyfirst.substack.com/p/genie-wants-to-leap).

right," it changes something, because they're independent of your framing. Their agreement costs them something: they considered it and chose to agree, not that they were designed to be helpful.

Someone who has built observability pipelines telling me the foundation holds would settle something the model's agreement can't.

AI agreement is cheap. Human agreement from the right person is not.

The second is scope judgment. Am I reaching too high or not high enough? Is an agentic nervous system too ambitious for a conference demo, or not ambitious enough to deserve a stage? I can't see that from inside it. An AI will help a person execute whatever direction they've chosen; it won't tell them the direction is wrong. That takes someone who has seen both failure modes: the plan that collapsed under its own weight and the one that never got anywhere, and can recognize, from the outside, which one the work is heading toward.

The agent will hold any direction. It can't tell you yours is wrong.

The third is the unknown unknown. The tool no one in the room is aware of. The approach that would make everything simpler but simply hasn't been encountered. If a framework already does what I built over the demo, the model and I are both searching for a name I don't know.

AI can surface unknown unknowns within the space it has been pointed at. Ask it "what am I missing here" and it will often find something useful. The harder gap is context-specific: the thing someone with direct experience of the situation would see that neither you nor the model would think to name. That requires someone looking in from outside the framing.

You can't prompt for what you don't know to name.

The fourth is teaching. This one has no JoustMania version. There is a difference between a thing being explained and a thing being taught. AI explains well. It can walk a person through a concept, surface the relevant literature, answer follow-up questions patiently. But teaching requires someone who recognizes when the learner doesn't know something yet and can hold them through that state until the understanding arrives. The scar tissue matters, and so does the presence of someone who notices they're there and knows what to do about it.

Research is a substitute for teaching, slower and lonelier.

The Spec Session is where the first three happen, before the agent runs.

The distributed async team loses these things first. The timezone gap, the Kanban board, the scoping documents that arrive as finished artifacts, all of it slowly drains the moments where

weighted validation happens, where someone challenges the scope, where an unknown unknown gets surfaced by someone who happened to know the thing no one else did.

And AI accelerates the drain through usefulness. The tool is faster, always available, never in a meeting, so the maker stops reaching for the person and starts reaching for the tool.

Until you have a working demo and no stage to test it against, and the question of whether the shape is right sits with you in a way it wouldn't if someone else had checked the architecture first.

Another person can provide all four, but a person isn't a structure. The structural answer is the team.

This isn't only the solo builder's problem. The same gap opened on a team a few chapters back, where the junior catching what the agent and I had both stopped seeing was supplying exactly this, a check against reality that no prompt produced. Take that junior out of the room, let everyone review alone against their own prompt, and the agreement gets cheap again.

The team is the environment where independent judgment can happen, not a delivery mechanism but the place where judgment gets tested. Where someone pushes back from a starting point that isn't yours, the one place an agent, working from your framing, can never stand.

AI is frictionless by design. It finds a way to make an instinct seem reasonable. It smooths the edges off the direction already chosen. That's useful for execution and dangerous for judgment.

The team has friction by nature. Different experience, different exposure, different mental models of where the system is going and why. That friction is the mechanism. The reason the team's judgment is different from the AI's agreement is precisely that the team doesn't have to agree.

A form of friction can be engineered from an AI: build the context to argue back, load it with counterarguments, instruct it to push back. But the pushback is still downstream of the maker's framing. The AI can challenge stated assumptions. The unstated ones are invisible to it.[2][3]

None of the four is a capability gap. Capability gaps close; that's what model releases do. These four are consequences of the one fact that doesn't: however capable the genie gets, it never gets

[2]Alibaba's Qwen team makes the same point formally in "The Verification Horizon: No Silver Bullet for Coding Agent Rewards" (2026, arxiv.org/abs/2606.26300): every verifier is only a proxy for human intent, never the intent itself.

[3]Anthropic's own harness engineers report the same from the inside: agents grading their own work reliably skew positive, and making a generator critical of its own output proved far less tractable than tuning a separate, skeptical evaluator — whose judgment, in turn, had to be calibrated against a human's. Notably, their multi-agent harness also converged on a spec-first ritual: generator and evaluator negotiating what "done" looks like before any code is written. Prithvi Rajasekaran, "Harness Design for Long-Running Application Development" (anthropic.com/engineering, March 2026).

to want. That's why the human doesn't move out of the loop as the model improves. We were never in it because we were better at something. We're in it because the loop is for us.

Understanding is only useful if it can be challenged. And the place where it gets challenged, by someone who doesn't have to agree, is the team.

The friction is the point.

Weighted validation, scope judgment, the unknown unknown, teaching.

Three of the four keep your judgment honest while you build. Teaching does something the other three don't. It is how the judgment survives the person who has it, and it is the first thing the agent takes. So where does the next person learn to be the one who doesn't have to agree, when the tool is always the faster answer?